SIMATS ENGINEERING

ASSESSMENT TOOL 2

CRYPTOGRAPHY AND NETWORK SECURITY

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA51

COURSE OUTCOME COVERED CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5) Scenario-Based : 20% QUESTIONS: Total Marks: 50

Annexure A

Code of Conduct: I _____J.Vishnu Vardhan(192324297)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A Scenario-based

1. Secure Data Transmission using Block Cipher A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext. A. Explain why this issue occurs. B. Suggest a more secure block cipher mode and justify your choice based on the scenario.
2. Choosing an Encryption Algorithm for Banking System A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish. A. Identify the most suitable algorithm for this scenario. B. Justify your choice based on security strength and efficiency.
3. Key Exchange in an Unsecured Network Two users want to establish a secure communication channel over a public network using Diffie-Hellman key exchange. However, the network is vulnerable to attacks. A. Explain how the key exchange works in this scenario. B. Identify a possible security threat and suggest a solution to mitigate it.
4. Public Key Cryptography for Email Security An organization plans to implement RSA for secure email communication. A. Describe how RSA can be used for encryption and decryption in this scenario. B. Explain how key distribution and management play a role in maintaining security.
5. Cryptography for IoT Devices A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC). A. Analyze the constraints of IoT devices. B. Recommend a suitable cryptographic approach and justify your answer.

RUBRICS FOR CALCULATION:

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context.	Shows good understanding with minor gaps.	Partial understanding; misses key aspects.	Misinterprets or fails to understand the scenario.
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved.	Identifies most key issues with minor omissions.	Identifies limited issues; some irrelevant points.	Unable to identify key issues.
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario.	Applies concepts with minor errors.	Limited or partially correct application.	Unable to apply relevant concepts.
Decision Making	20%	Makes well-reasoned,	Makes reasonable decisions	Makes weak decisions	No clear decisions made or

		logical decisions with strong rationale.	with some justificatio n.	with limited justification .	rationale provided.
Clarity of Response	15%	Response is clear, structured, and logically presented.	Generally clear with minor issues.	Somewhat unclear or poorly structured.	Disorganiz ed and difficult to understand .

QUESTION 1: SECURE DATA TRANSMISSION USING BLOCK CIPHER

Scenario: A company is transmitting repeated financial data blocks over a network. Initially, it used Electronic Codebook (ECB) mode and noticed patterns in the ciphertext.

Introduction

Cryptography protects information by converting readable data, called plaintext, into an unreadable form called ciphertext. A block cipher encrypts data in fixed-size blocks using a secret key. The algorithm and key are important, but the mode of operation also determines how multiple blocks are processed. In a financial system, transaction information may contain repeated values, account fields, standard message formats, and similar records. Therefore, choosing an appropriate mode is essential to protect confidentiality and integrity.

ECB is one of the simplest block cipher modes. However, it encrypts every block independently. This design makes it easy to implement but can reveal patterns when the plaintext contains repeated blocks. The following analysis

explains why the problem occurs and recommends a safer mode for the company's financial transmissions.

A. Why does the pattern issue occur in ECB mode?

1. Working principle of ECB

In ECB mode, plaintext is divided into fixed-size blocks. Each block is encrypted separately using the same secret key. If P1, P2, and P3 represent plaintext blocks and E(K, P) represents encryption using key K, then the ciphertext blocks are C1 = E(K, P1), C2 = E(K, P2), and C3 = E(K, P3). The encryption of one block does not depend on the contents of the previous block.

Consequently, when two plaintext blocks are identical and the same key is used, their ciphertext blocks are identical. This deterministic property is the central weakness of ECB for data containing repetition.

2. Repeated plaintext produces repeated ciphertext

Financial records often have repeated fields such as transaction types, fixed labels, currency codes, or recurring payment values. If identical full plaintext blocks occur more than once, ECB produces matching ciphertext blocks. An observer who captures the encrypted traffic cannot necessarily read the actual values, but can notice that particular blocks recur and determine where the data has similar structure.

Illustrative example:

Plaintext blocks: PAY100 | PAY200 | PAY100

ECB ciphertext: X7A2 | K9B4 | X7A2

The first and third plaintext blocks are the same, so their ciphertext blocks match. These symbols are only illustrative and are not real encryption output.

3. Why pattern leakage is a security concern

1. Information leakage: Repeated ciphertext blocks disclose relationships within the message, even though the key is unknown.
2. Traffic analysis: An observer may compare encrypted messages and identify recurring structures or repeated activity.
3. Financial privacy risk: Patterns may reveal that similar types of transactions or records occur repeatedly.

4. No built-in integrity protection: ECB does not detect whether ciphertext blocks have been changed, removed, or rearranged.
5. No randomization across blocks: ECB does not use an initialization vector (IV) or nonce to make repeated plaintext encrypt differently.

ECB does not automatically reveal every plaintext value, and the amount of information leaked depends on the data structure. Nevertheless, the pattern leakage is enough to make ECB unsuitable for most sensitive, structured data transmissions.

4. Other limitations of ECB

Each block can be encrypted or decrypted independently, which may appear convenient. However, that same independence means an attacker can manipulate blocks without the mode itself detecting the change. ECB also requires data to be processed in complete blocks, so padding may be necessary for messages whose lengths are not block-aligned. Correct padding does not solve the pattern-leakage problem.

B. Recommend a more secure block cipher mode and justify the choice

Recommended mode: AES-GCM

The company should use AES-GCM (Advanced Encryption Standard in Galois/Counter Mode), provided it is implemented correctly. AES-GCM is an authenticated encryption mode: it protects the confidentiality of the data and generates an authentication tag to help detect tampering. Unlike ECB, AES-GCM does not encrypt each plaintext block independently in a way that exposes identical-block patterns.

1. Main advantages of AES-GCM

- Confidentiality: AES-GCM encrypts the financial data so that an unauthorized observer cannot read the plaintext without the key.
- Integrity: An authentication tag allows the receiver to detect unauthorized modifications to the ciphertext or authenticated associated data.
- Authentication of protected data: Verification of the tag confirms that the protected data has not been altered, assuming correct key use and secure implementation.
- Efficiency: AES-GCM is widely supported and can be efficient on modern processors with AES acceleration.

- Suitable for network communication: It is commonly used in secure communication protocols and APIs when correctly configured.

2. How AES-GCM works

6. The sender prepares the financial transaction data that needs protection.
7. The sender selects the correct secret key and generates a unique nonce for that key.
8. AES-GCM encrypts the plaintext using the key and nonce.
9. The encryption process produces ciphertext and an authentication tag. Additional authenticated data, such as selected protocol headers, may also be protected against modification without being encrypted.
10. The sender transmits the ciphertext, nonce, and authentication tag to the receiver over the network.
11. The receiver uses the same key and supplied nonce to verify the authentication tag.
12. Only if verification succeeds does the receiver accept and decrypt the data. If verification fails, the message must be rejected.

3. Important implementation requirements

AES-GCM must be used with a unique nonce for every encryption under the same key. Reusing a nonce with the same key can seriously weaken confidentiality and integrity. The system should use a reliable nonce-generation or allocation strategy, protect keys from unauthorized access, and rotate keys according to a documented policy. Authentication tags must be verified before the application acts on decrypted data. The company should use a well-maintained cryptographic library rather than designing its own encryption code.

4. ECB and AES-GCM comparison

Feature	ECB Mode	AES-GCM
Processing	Each block is encrypted independently	Counter-mode encryption with authentication
Repeated patterns	Identical plaintext blocks produce identical ciphertext blocks	Avoids ECB-style pattern leakage when used correctly

Integrity	Not provided	Authentication tag detects tampering
Nonce / IV	Not used	Requires a unique nonce per key
Financial data suitability	Not recommended	Suitable when correctly implemented

5. Additional security measures for the company

- Use TLS to protect data while it travels between authorized systems.
- Store encryption keys in a protected key-management service or hardware security module where appropriate.
- Apply access control so only authorized services can use the keys.
- Log and monitor suspicious access and failed authentication checks.
- Keep cryptographic libraries and servers updated, and review configurations regularly.
- Avoid logging plaintext financial data or secret keys.

Conclusion

The company's use of ECB mode exposes repeated patterns because identical plaintext blocks produce identical ciphertext blocks under the same key. This can leak structural information about financial transactions and ECB provides no integrity protection. AES-GCM is a better option because it offers confidentiality and integrity together. The company must use unique nonces, protect its keys, verify authentication tags, and follow secure implementation practices.

QUESTION 5: CRYPTOGRAPHY FOR IoT DEVICES

Scenario: A smart home system uses low-power Internet of Things (IoT) devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

Introduction

The Internet of Things (IoT) connects physical devices such as smart locks, temperature sensors, lighting systems, door sensors, cameras, and home gateways to a network. These devices exchange commands and information and may control important household functions. If communication is not protected, an attacker may attempt to read messages, impersonate a device, or modify commands.

Public-key cryptography supports secure authentication, digital signatures, and key establishment. RSA and ECC are two public-key approaches, but they have different resource requirements. Since many IoT devices have limited processing power, memory, battery capacity, and network bandwidth, the designer must select an approach that balances security and efficiency.

A. Analyze the constraints of IoT devices

1. Limited processing power

Many IoT devices use low-cost microcontrollers rather than powerful computer processors. Public-key operations require mathematical calculations that can be demanding for small devices. Expensive operations may increase response time and reduce the number of operations a device can perform. RSA private-key operations can be costly, while ECC operations are often more practical at comparable security levels, depending on the hardware and implementation.

2. Limited memory and storage

IoT devices may have very small amounts of RAM and flash storage. Cryptographic libraries, certificates, keys, and protocol messages all consume memory. RSA typically requires larger keys than ECC for comparable security, increasing the size of public keys, certificates, and some transmitted data. ECC's smaller keys can reduce storage requirements and communication overhead.

3. Battery and energy limitations

Some smart sensors operate on batteries and may need to work for months or years without replacement. Cryptographic computations, wireless communication, and frequent handshakes all consume energy. An efficient cryptographic design can help extend battery life. ECC often provides an

advantage because it uses smaller keys and can reduce the amount of data transmitted, although actual energy use must be measured on the target device.

4. Limited communication bandwidth

IoT networks may use low-power wireless technologies with small packet sizes or limited data rates. Large public keys and certificates can require more packets, increasing latency and energy consumption. ECC's shorter keys can help reduce this overhead. Protocol designers should also minimize unnecessary handshakes and avoid transmitting the same certificate repeatedly when safe caching is supported.

5. Security and privacy requirements

Smart home devices may handle private information or control physical access. A smart lock must accept commands only from authorized users; a sensor must not accept forged readings; and a gateway must communicate with genuine devices. The cryptographic system should provide authentication, confidentiality, integrity, and protection against replay attacks. Cryptography alone is not enough: secure boot, signed firmware updates, access control, and timely patching are also necessary.

6. Large number of devices and key management

A home may contain many devices from different manufacturers. Each device needs a secure identity and a method for obtaining, storing, updating, and revoking keys or certificates. If one device is compromised, the system should be able to revoke its credentials without replacing every device. The key-management design should scale as new devices are added and old devices are removed.

7. Cost and compatibility

Low-cost devices may not include hardware acceleration for every cryptographic algorithm. Some devices may have existing protocol or certification requirements that influence the choice. A designer should test supported cryptographic libraries, processing time, memory use, energy consumption, and compatibility before deployment. A theoretically efficient algorithm is not automatically the best choice if the device cannot implement it safely.

B. Recommend a suitable cryptographic approach and justify your answer

Recommendation: ECC for public-key operations

For a typical low-power smart home system, ECC is generally the preferred option over RSA because it can provide comparable security with much smaller keys. Smaller keys reduce storage and network overhead, and ECC-based operations can be efficient on constrained hardware. This recommendation assumes the device and communication protocol support a secure, well-reviewed ECC implementation.

1. Reasons for choosing ECC

13. Smaller key sizes: ECC achieves strong security with shorter keys than RSA at comparable security levels.
14. Reduced communication overhead: Shorter keys and related public-key data can reduce the amount of information transmitted over low-bandwidth IoT networks.
15. Efficient key establishment: Elliptic-curve Diffie–Hellman methods can help two devices establish a shared secret without sending the secret itself over the network.
16. Support for authentication: ECC-based digital signatures can help verify that a device or update comes from an authorized source.
17. Suitable for constrained devices: ECC is often a good fit for systems with limited memory, processing resources, and battery capacity.

For perspective, a properly configured 256-bit elliptic-curve key provides security broadly comparable to a 3072-bit RSA key. These are approximate security-level comparisons, not a claim that every ECC implementation is faster on every device.

2. Suggested secure architecture for a smart home

A practical design should not use RSA or ECC to encrypt every message directly. Public-key cryptography is generally used to authenticate devices and establish or protect a session key. A symmetric algorithm is then used for routine communication because symmetric encryption is typically much faster.

Step 1: Device identity and authentication

Each device should have a unique identity and a protected private key. The device may use an ECC digital-signature scheme such as ECDSA, or another

modern signature scheme supported by the platform, to prove its identity. The corresponding public key or certificate is validated by the gateway or other communicating device.

Step 2: Establish a shared session key

The device and the gateway can use an authenticated elliptic-curve Diffie–Hellman protocol, such as ECDHE combined with suitable authentication, to establish a shared secret. The protocol must authenticate the parties; otherwise, an attacker may attempt to place themselves between them. The shared secret is passed through a suitable key-derivation function to create session keys.

Step 3: Encrypt routine messages

Once the session key is established, the devices can use an authenticated symmetric encryption algorithm such as AES-GCM to protect commands and sensor readings. This provides confidentiality and integrity with lower computational cost for repeated data exchange than performing public-key encryption for every message.

Step 4: Prevent replay and unauthorized commands

Messages should include appropriate sequence numbers, counters, nonces, or timestamps within the authenticated protocol design to prevent old valid messages from being replayed. A smart lock should verify that commands are authenticated, fresh, and authorized before unlocking.

Step 5: Secure key storage and updates

Private keys should be stored in secure hardware or protected storage where available. Devices should support credential revocation, key rotation, and authenticated firmware updates. Firmware updates must be verified before installation, and unused services should be disabled to reduce the attack surface.

3. Example: Securing a smart door lock

Consider a smart door lock controlled by an authorized mobile application through a home gateway. When the application connects, it validates the gateway's identity. The application and gateway perform an authenticated ECC-based key exchange and derive session keys. Commands are then protected using AES-GCM. When an unlock command arrives, the lock or gateway

verifies the authentication tag and checks that the command is fresh and authorized. If the message has been changed or replayed, the system rejects it.

This design protects the communication channel while avoiding the cost of applying public-key operations to every command. The system must still enforce strong user authentication, secure device pairing, and safe recovery procedures.

4. RSA and ECC comparison for IoT

Factor	RSA	ECC
Security basis	Integer factorization	Elliptic-curve discrete logarithm
Key size at comparable security	Larger	Smaller
Communication overhead	Generally higher	Generally lower
Resource use on constrained devices	Can be demanding	Often more suitable
Best fit in this scenario	Where compatibility or existing infrastructure requires it	Typically preferred for low-power IoT devices

5. Limitations and implementation considerations

ECC is not automatically secure simply because it uses shorter keys. The designer must choose approved curves and algorithms, use a trusted cryptographic library, generate keys securely, and validate public keys and certificates correctly. Poor random-number generation, insecure private-key storage, or missing authentication can undermine either RSA or ECC. The system should also receive security updates throughout its supported lifetime.

Some legacy devices or protocols may support RSA but not the required ECC algorithms. In such cases, compatibility must be evaluated, and an upgrade may

be necessary. Testing on actual hardware should measure handshake time, RAM and flash use, packet size, and energy consumption before final deployment.

Conclusion

IoT devices face limitations in processing power, memory, battery life, bandwidth, and long-term maintenance. ECC is generally a better public-key choice than RSA for low-power smart home devices because it offers comparable security with smaller keys and lower communication overhead. A robust design should use authenticated ECC-based key establishment and device authentication, then use AES-GCM with derived session keys for routine data encryption. Secure key storage, replay protection, firmware updates, and lifecycle management are essential to complete the solution.